

```
(base) jishnu@jishnu-Lenovo-G50-70:~/Desktop$ gcc -o helloc -fopenmp hello.c
(base) jishnu@jishnu-Lenovo-G50-70:~/Desktop$ export OMP_NUM_THREADS=2
(base) jishnu@jishnu-Lenovo-G50-70:~/Desktop$ ./helloc
Hello World from thread = 0
Number of threads = 2
Hello World from thread = 1
(base) jishnu@jishnu-Lenovo-G50-70:~/Desktop$
```

Figure 1: Getting the thread numbers

```
(base) jishnu@jishnu-Lenovo-G50-70:~/Documents/sem 5/pdc lab/lab2$ gcc -o filename -fopenmp lab2.c
(base) jishnu@jishnu-Lenovo-G50-70:~/Documents/sem 5/pdc lab/lab2$ ./filename
CPU:0 Thread:2 Value8 at i4
CPU:0 Thread:2 Value10 at i5
CPU:0 Thread:3 Value12 at i6
CPU:0 Thread:3 Value14 at i7
CPU:0 Thread:0 Value0 at i0
CPU:0 Thread:0 Value2 at i1
CPU:0 Thread:0 Value16 at i8
CPU:0 Thread:0 Value18 at i9
CPU:0 Thread:1 Value4 at i2
CPU:0 Thread:1 Value6 at i3
```

Figure 2: Parallelising task

```
nthreads); } }
}
```

In this code we have used a few functions that are new.

The `omp_get_thread_num()` function helps us to extract the thread number. Then we can give it conditions — if the thread number is this, do this, etc, using *if-else* conditions. I asked all the threads to print the thread number and one thread to print the number of threads that are there. This is done accurately by the code as seen in the output in Figure 1.

Now let us solve a little more complicated problem. Let us add the corresponding elements of two arrays. This is just the kind of problem we want to solve using parallel programming because each element is added separately. There is no reason for us to add these one by one. So one thread takes up the task of adding one of the corresponding elements at a time. You can assign the threads based on the number of cores your laptop has so that

your machine is more efficient.

Let us do this now.

Paste the following code in a text editor:

```
#include <omp.h>
#define _GNU_SOURCE
#include<stdio.h>
#include <sched.h>
int main ()
{
    int a[10]={0,1,2,3,4,5,6,7,8,9};
    int b[10]={0,1,2,3,4,5,6,7,8,9}; int
    sum[10];
    int i;
    int p=sched_getcpu();
    #pragma omp parallel for
    schedule(static,2)
    for (i=0; i < 10; i++)
    {
        sum[i] = a[i] + b[i];
        printf("CPU:%d\tThread:%d\tValue%d\t
```

```
at i %d\n",p,omp_get_thread_
num(),sum[i],i);
} }
```

Here we give ‘status’ and ‘parallel’ as arguments, and a *for* loop after that. What this does is that each of the iterations is given to a separate thread and then run. In the end the results can be accumulated to get a final result as shown in Figure 2.

That is all; you now have a basic understanding of parallel programming and how to use OpenMP for the same. There are so many more features that OpenMP offers like reduction, sections, and more. I would recommend you go deeper into these as that is when it gets to be more fun. You can also come up with more parallelisation, and make all your processes more efficient. **END** 🐧

By: Jishnu Saurav Mittapalli

The author is a full-stack web developer, interested in modern technologies like artificial intelligence and machine learning. He is currently working in the field of natural language processing.